UNIVERSIDAD POLITECNICA DE ATLAUTLA

Sesion 1: “Aplicaciones y casos de estudio de inteligencia artificial”.

Fundamentos de Inteligencia Artificial

Alumno:

Castro Soriano Ronaldo Malcom

Profesor:

Ing. García Lara Javier.

Atlautla, Estado de México a 18 de octubre de 2026.

1. Planteamiento de la actividad

El objetivo de esta actividad es formular dos situaciones como problemas de búsqueda, representar sus espacios de estados mediante grafos dirigidos y ponderados y resolver las rutas de menor costo utilizando Python y NetworkX. Los casos seleccionados son el traslado de una ambulancia desde su base hasta un hospital y la distribución de artesanías desde un taller hasta un punto de entrega.

En Inteligencia Artificial, un problema de búsqueda se formula a partir de un estado inicial, las acciones disponibles, un modelo de transición, una prueba de meta y un costo de camino. Esta representación permite comparar alternativas y seleccionar una solución de acuerdo con un criterio definido (Russell & Norvig, 2021).

1.1. Representación del espacio de búsqueda

Se utiliza un grafo dirigido G = (V, E). El conjunto V contiene los nodos o estados; el conjunto E contiene las transiciones permitidas entre los estados. Cada arista tiene un peso no negativo. En el primer caso, el peso representa minutos de recorrido; en el segundo, kilómetros estimados.

El grafo es dirigido porque cada flecha indica un movimiento permitido desde un origen hacia un destino. No se supone que exista automáticamente una conexión de regreso. Para encontrar el camino de costo mínimo se utiliza el algoritmo de Dijkstra, apropiado cuando los pesos de las aristas son no negativos (NetworkX Developers, s. f.-b).

1.2. Objetivo general

Construir y evaluar dos espacios de búsqueda dirigidos y ponderados, generar los estados alcanzables desde un punto inicial y calcular una ruta de costo mínimo hasta una meta mediante NetworkX.

Nota sobre los datos: los tiempos y las distancias usados en ambos ejemplos son datos didácticos simulados. No provienen de mapas, mediciones oficiales ni sistemas de tráfico en tiempo real, por lo que no deben emplearse para tomar decisiones operativas.

2. Caso 1. Ruta de una ambulancia hacia un hospital

2.1. Planteamiento del problema

Una unidad de emergencia debe trasladarse desde su base hasta un hospital. Existen varios cruces posibles y los tramos tienen tiempos de recorrido distintos. El objetivo consiste en encontrar una secuencia de tramos permitidos que reduzca al mínimo el tiempo total estimado.

2.2. Formulación del problema

Estado: ubicación actual de la ambulancia, representada por un nodo del grafo.

Estado inicial: Base de ambulancias.

Acciones: avanzar por una arista saliente hacia un cruce o destino conectado.

Modelo de transición: desde un nodo u se puede llegar a un nodo v si existe la arista dirigida (u, v).

Prueba de meta: el estado actual es Hospital.

Costo de acción: peso de la arista, interpretado como minutos de recorrido.

Costo del camino: suma de los minutos de todos los tramos recorridos.

Criterio de solución: encontrar la ruta desde Base hasta Hospital con el menor tiempo total.

2.3. Estados y conexiones del grafo

Los estados son Base, Cruce A, Cruce B, Cruce C, Cruce D, Cruce E y Hospital. El grafo contiene siete nodos y once aristas dirigidas.

Las conexiones y sus costos son los siguientes:

Base → Cruce A: 4 minutos.

Base → Cruce B: 7 minutos.

Cruce A → Cruce C: 5 minutos.

Cruce A → Cruce D: 8 minutos.

Cruce B → Cruce D: 3 minutos.

Cruce B → Cruce E: 6 minutos.

Cruce C → Hospital: 6 minutos.

Cruce D → Cruce C: 2 minutos.

Cruce D → Hospital: 7 minutos.

Cruce E → Cruce D: 2 minutos.

Cruce E → Hospital: 8 minutos.

2.4. Diagrama del espacio de búsqueda

En el diagrama, las flechas muestran la dirección de cada transición y los números indican los minutos simulados del tramo. La ruta de costo mínimo aparece resaltada en rojo.

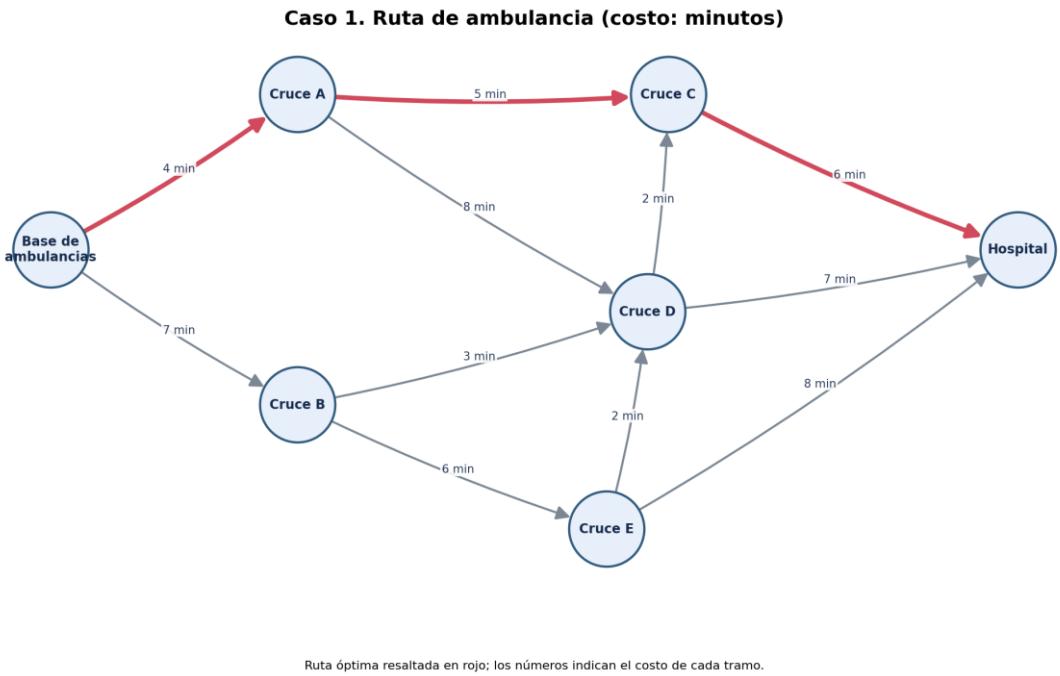


Figura 1. Grafo dirigido y ponderado de la ruta de ambulancia. Elaboración propia con datos simulados.

2.5. Procedimiento y solución

Primero se construye el grafo con los siete estados y los once tramos. Después, el generador de estados recorre las aristas salientes desde Base y registra cada nodo alcanzable una sola vez. A continuación, se ejecuta Dijkstra con Base como origen, Hospital como meta y el atributo weight como costo. Por último, el programa suma los pesos de las aristas seleccionadas para comprobar el resultado.

Ruta obtenida: Base → Cruce A → Cruce C → Hospital.

Costo total: 4 + 5 + 6 = 15 minutos simulados.

Una alternativa es Base → Cruce B → Cruce D → Hospital, cuyo costo es 7 + 3 + 7 = 17 minutos. Por lo tanto, dentro del grafo definido, la ruta de 15 minutos es la de menor costo.

Interpretación: la respuesta es óptima respecto de los pesos definidos en el ejemplo, pero no necesariamente respecto del tráfico real. Para construir un sistema operativo serían necesarios datos viales verificados, sentidos de circulación, restricciones, tráfico actualizado y validación de seguridad.

3. Caso 2. Distribución de artesanías

3.1. Planteamiento del problema

Un taller artesanal necesita trasladar sus productos hasta un punto de entrega. El recorrido puede pasar por cruces, un centro de empaque y un mercado local. Cada tramo tiene una

distancia estimada. El objetivo es reducir la distancia total de un traslado entre el taller y el destino.

La distribución de última milla es un problema relevante en logística porque la elección de recorridos puede influir en los costos y en la eficiencia de las entregas. La investigación de Rodrigues et al. (2025) estudia la planeación de distribución de última milla en zonas rurales considerando rutas y múltiples restricciones. En esta actividad el escenario se simplifica a un solo envío y un único punto de entrega.

3.2. Formulación del problema

Estado: ubicación actual de los productos durante el recorrido.

Estado inicial: Taller artesanal.

Acciones: trasladar los productos por una conexión dirigida disponible.

Modelo de transición: el estado cambia de u a v cuando la arista dirigida (u, v) está presente.

Prueba de meta: se alcanza el Punto de entrega.

Costo de acción: peso de la arista, interpretado como kilómetros simulados.

Costo del camino: suma de las distancias asignadas a todos los tramos recorridos.

Criterio de solución: encontrar la ruta de distancia mínima desde Taller hasta Entrega.

3.3. Estados y conexiones del grafo

Los estados son Taller artesanal, Cruce Norte, Cruce Sur, Centro de empaque, Mercado local y Punto de entrega. El grafo contiene seis nodos y diez aristas dirigidas.

Las conexiones y sus costos son los siguientes:

Taller → Cruce Norte: 3 kilómetros.

Taller → Cruce Sur: 5 kilómetros.

Cruce Norte → Cruce Sur: 4 kilómetros.

Cruce Norte → Centro de empaque: 5 kilómetros.

Cruce Sur → Centro de empaque: 2 kilómetros.

Cruce Sur → Mercado local: 7 kilómetros.

Centro de empaque → Mercado local: 3 kilómetros.

Centro de empaque → Punto de entrega: 7 kilómetros.

Mercado local → Punto de entrega: 1 kilómetro.

Cruce Norte → Punto de entrega: 14 kilómetros.

3.4. Diagrama del espacio de búsqueda

Las flechas representan las conexiones permitidas y sus etiquetas expresan kilómetros simulados. La ruta de menor costo se resalta en rojo.

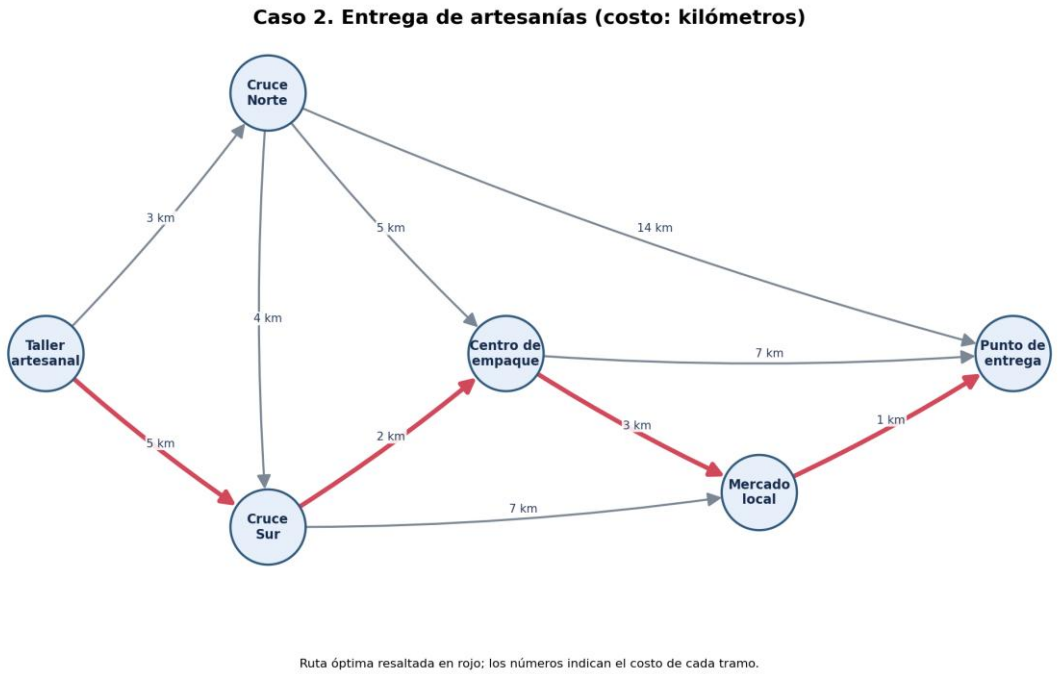


Figura 2. Grafo dirigido y ponderado para distribuir artesanías. Elaboración propia con datos simulados.

3.5. Procedimiento y solución

Se construye el grafo con seis estados y diez aristas ponderadas. El generador recorre los nodos alcanzables a partir del Taller. Después se aplica Dijkstra con Taller como origen y Entrega como meta. El programa comprueba que los pesos sean numéricos y no negativos y valida el costo calculado mediante la suma de las aristas de la ruta encontrada.

Ruta obtenida: Taller → Cruce Sur → Centro de empaque → Mercado local → Punto de entrega.

Costo total: 5 + 2 + 3 + 1 = 11 kilómetros simulados.

La alternativa Taller → Cruce Norte → Centro de empaque → Mercado local → Punto de entrega suma 3 + 5 + 3 + 1 = 12 kilómetros. En el modelo, la ruta seleccionada es un kilómetro más corta.

Interpretación: el resultado representa un solo envío y una única meta. Un problema logístico real podría requerir restricciones adicionales, como capacidad de carga, horarios, varios destinos, combustible, estado de los caminos y regreso al taller. Estos factores no forman parte del modelo básico.

4. Generador de estados y análisis de complejidad

4.1. Generación de estados

El programa incluye la función generar_estados(grafo, estado_inicial), implementada mediante una cola FIFO y una búsqueda en anchura (BFS). La función comienza con el estado inicial, lo

registra y revisa sus sucesores dirigidos. Cada sucesor que no se haya visitado se agrega a la cola. Así se generan los estados alcanzables desde el origen sin repetir nodos, incluso si el grafo contuviera ciclos.

El uso de una colección de estados visitados evita volver a procesar un nodo. La función recorre las aristas salientes, lo que respeta el carácter dirigido de los grafos.

4.2. Complejidad temporal y espacial

Sea |V| el número de nodos y |E| el número de aristas. El análisis se realiza en función del tamaño del grafo, no solamente del número de nodos de estos dos ejemplos.

Construcción del grafo: tiempo O(|V| + |E|), porque se registran los nodos y las conexiones. El almacenamiento del grafo requiere O(|V| + |E|) espacio.

Generación de estados mediante BFS: tiempo O(|V| + |E|), ya que cada nodo alcanzable se procesa una vez y se revisan las aristas salientes. El espacio adicional para la cola y el conjunto de visitados es O(|V|).

Algoritmo de Dijkstra: con una cola de prioridad basada en heap binario, el tiempo es O((|V| + |E|) log |V|). El uso de memoria para la estructura de grafo, las distancias y la cola se expresa como O(|V| + |E|) en una cota general.

Validación de pesos: tiempo O(|E|), porque se revisa cada arista para comprobar que su peso sea numérico y no negativo.

Verificación del costo: tiempo O(L), donde L es la cantidad de aristas en la ruta encontrada, porque se suman los pesos de sus tramos.

La complejidad de Dijkstra depende de la estructura utilizada para seleccionar el nodo con menor distancia provisional. La documentación de NetworkX indica que la implementación utiliza un heap binario y describe la complejidad general para rutas con pesos no negativos (NetworkX Developers, s. f.-b).

4.3. Resultados de la ejecución y verificación

Caso 1, ambulancia: tipo de grafo DiGraph ponderado; 7 nodos; 11 aristas; 7 estados alcanzables desde Base; ruta Base → Cruce A → Cruce C → Hospital; costo de 15 minutos simulados; costo verificado por la suma 4 + 5 + 6; meta alcanzada: sí.

Caso 2, artesanías: tipo de grafo DiGraph ponderado; 6 nodos; 10 aristas; 6 estados alcanzables desde Taller; ruta Taller → Cruce Sur → Centro de empaque → Mercado local → Punto de entrega; costo de 11 kilómetros simulados; costo verificado por la suma 5 + 2 + 3 + 1; meta alcanzada: sí.

La ejecución del archivo Python produjo los dos diagramas y mostró las rutas y sus costos. La verificación confirma la consistencia interna del modelo; no demuestra que los tiempos o las distancias sean representativos de una operación real.

5. Conclusiones

Los dos problemas se formularon identificando el estado inicial, las acciones permitidas, las transiciones, la meta y el costo del camino. Los grafos dirigidos representan movimientos disponibles en un sentido, mientras que los pesos permiten expresar un criterio de optimización: tiempo total para el traslado de la ambulancia y distancia total para la entrega de artesanías.

El algoritmo de Dijkstra encontró una ruta de 15 minutos simulados desde la Base hasta el Hospital y una ruta de 11 kilómetros simulados desde el Taller hasta el Punto de entrega. En ambos casos, el generador identificó todos los estados alcanzables y el programa verificó los costos mediante la suma de los pesos. El análisis de complejidad permite estimar cómo crece el esfuerzo de búsqueda cuando aumenta el tamaño de la red.

La principal limitación es el uso de grafos pequeños y costos simulados. Una implementación real necesitaría obtener y actualizar datos de la red vial, validar los sentidos y restricciones y tomar en cuenta los factores particulares del servicio. Por tanto, la actividad demuestra el modelado de problemas de búsqueda, pero no sustituye una aplicación de navegación o logística en producción.

Anexo A. Código Python comentado

El siguiente listado corresponde al archivo actividad_21_22_busqueda.py. Requiere Python y las bibliotecas NetworkX y Matplotlib. Para instalarlas, ejecutar: python -m pip install networkx matplotlib. Para ejecutar el programa, guardar el archivo y escribir: python actividad_21_22_busqueda.py.

```
"""Actividad 2.1 y 2.2: problemas y espacios de búsqueda con NetworkX.
Los costos de las aristas son datos didácticos simulados. No representan
mediciones oficiales de tránsito ni distancias tomadas de un mapa real.
Requisitos:
    python -m pip install networkx matplotlib
Ejecución:
    python actividad_21_22_busqueda.py
"""
from collections import deque
from pathlib import Path
from typing import Iterator
import matplotlib.pyplot as plt
import networkx as nx
CARPETA_SALIDA = Path(__file__).resolve().parent / "diagramas"
def crear_problema_ambulancia() -> tuple[nx.DiGraph, dict, dict, str, str, str]:
    """Construye el espacio de rutas de una ambulancia.
    Cada nodo representa una ubicación/intersección y cada arista dirigida,
    un tramo transitable. El peso se interpreta como minutos de recorrido.
    """
    grafo = nx.DiGraph(nombre="Ruta de ambulancia")
    etiquetas = {
        "Base": "Base de\nambulancias",
        "A": "Cruce A",
        "B": "Cruce B",
        "C": "Cruce C",
        "D": "Cruce D",
        "E": "Cruce E",
        "Hospital": "Hospital",
    }
    grafo.add_nodes_from(etiquetas)
    # (origen, destino, minutos): cada conexión se puede recorrer en el
    # sentido de la flecha; no se presupone que la calle permita volver.
```

```
    tramos = [
        ("Base", "A", 4),
        ("Base", "B", 7),
        ("A", "C", 5),
        ("A", "D", 8),
        ("B", "D", 3),
        ("B", "E", 6),
        ("C", "Hospital", 6),
        ("D", "C", 2),
        ("D", "Hospital", 7),
        ("E", "D", 2),
        ("E", "Hospital", 8),
    ]
    grafo.add_weighted_edges_from(tramos, weight="weight")
    posiciones = {
        "Base": (-0.2, 1.0),
        "A": (1.0, 2.0),
        "B": (1.0, 0.0),
        "C": (2.8, 2.0),
        "D": (2.7, 0.6),
        "E": (2.5, -0.8),
        "Hospital": (4.5, 1.0),
    }
    return grafo, etiquetas, posiciones, "Base", "Hospital", "min"
def crear_problema_artesanias() -> tuple[nx.DiGraph, dict, dict, str, str, str]:
    """Construye el espacio de rutas de entrega de artesanías.
    Cada nodo es un punto de paso o entrega y el peso indica kilómetros
    estimados para el ejemplo académico.
    """
    grafo = nx.DiGraph(nombre="Distribución de artesanías")
    etiquetas = {
        "Taller": "Taller\nartesanal",
        "Norte": "Cruce\nNorte",
        "Sur": "Cruce\nSur",
        "Empaque": "Centro de\nempaque",
        "Mercado": "Mercado\nlocal",
        "Entrega": "Punto de\nentrega",
    }
    grafo.add_nodes_from(etiquetas)
    # (origen, destino, kilómetros). Los sentidos reflejan las conexiones
    # permitidas dentro de este pequeño escenario de ejemplo.
    tramos = [
        ("Taller", "Norte", 3),
        ("Taller", "Sur", 5),
        ("Norte", "Sur", 4),
        ("Norte", "Empaque", 5),
        ("Sur", "Empaque", 2),
        ("Sur", "Mercado", 7),
        ("Empaque", "Mercado", 3),
        ("Empaque", "Entrega", 7),
        ("Mercado", "Entrega", 1),
        ("Norte", "Entrega", 14),
    ]
    grafo.add_weighted_edges_from(tramos, weight="weight")
    posiciones = {
        "Taller": (-0.2, 1.0),
        "Norte": (1.2, 2.5),
        "Sur": (1.2, 0.0),
        "Empaque": (2.7, 1.0),
        "Mercado": (4.3, 0.2),
        "Entrega": (5.9, 1.0),
    }
    return grafo, etiquetas, posiciones, "Taller", "Entrega", "km"
def generar_estados(grafo: nx.DiGraph, estado_inicial: str) -> Iterator[str]:
    """Genera los estados alcanzables desde el estado inicial mediante BFS.
    Se recorren únicamente las aristas salientes, porque el grafo es dirigido.
    Cada estado se emite una sola vez para evitar ciclos infinitos.
    """
    if estado_inicial not in grafo:
        raise ValueError(f"El estado inicial {estado_inicial!r} no existe en el grafo.")
```

```
    frontera = deque([estado_inicial])
    visitados = {estado_inicial}
    while frontera:
        estado_actual = frontera.popleft()
        yield estado_actual
        for siguiente in grafo.successors(estado_actual):
            if siguiente not in visitados:
                visitados.add(siguiente)
                frontera.append(siguiente)
def validar_pesos(grafo: nx.DiGraph) -> None:
    """Revisa que todas las aristas tengan peso numérico no negativo."""
    for origen, destino, datos in grafo.edges(data=True):
        peso = datos.get("weight")
        if not isinstance(peso, (int, float)) or peso < 0:
            raise ValueError(
                f"Peso inválido en la arista {origen!r} -> {destino!r}: {peso!r}"
            )
def resolver_problema(
    grafo: nx.DiGraph, inicial: str, meta: str
) -> tuple[list[str], float, list[str]]:
    """Genera los estados y obtiene la ruta de menor costo con Dijkstra."""
    validar_pesos(grafo)
    estados = list(generar_estados(grafo, inicial))
    if meta not in estados:
        raise nx.NetworkXNoPath(f"No existe una ruta dirigida de {inicial} a {meta}.")
    # Dijkstra es apropiado porque todos los pesos del modelo son no negativos.
    ruta = nx.dijkstra_path(grafo, source=inicial, target=meta, weight="weight")
    costo = nx.dijkstra_path_length(
        grafo, source=inicial, target=meta, weight="weight"
    )
    # Verificación independiente: suma manual de los pesos de la ruta obtenida.
    costo_verificado = sum(
        grafo[u][v]["weight"] for u, v in zip(ruta, ruta[1:])
    )
    if costo != costo_verificado:
        raise AssertionError("El costo calculado no coincide con la suma de las aristas.")
    return ruta, costo, estados
def dibujar_grafo(
    grafo: nx.DiGraph,
    etiquetas: dict,
    posiciones: dict,
    ruta: list[str],
    unidad: str,
    archivo: Path,
    titulo: str,
) -> None:
    """Guarda un diagrama dirigido y ponderado; resalta la ruta óptima."""
    archivo.parent.mkdir(parents=True, exist_ok=True)
    aristas_ruta = set(zip(ruta, ruta[1:]))
    colores_aristas = [
        "#D1495B" if (u, v) in aristas_ruta else "#7B8794"
        for u, v in grafo.edges()
    ]
    anchos_aristas = [
        3.0 if (u, v) in aristas_ruta else 1.4 for u, v in grafo.edges()
    ]
    plt.figure(figsize=(11, 6.3), dpi=200)
    nx.draw_networkx_nodes(
        grafo,
        posiciones,
        node_size=2450,
        node_color="#E7F0FA",
        edgecolors="#315A7D",
        linewidths=1.5,
    )
    nx.draw_networkx_labels(
        grafo,
        posiciones,
        labels=etiquetas,
        font_size=8.5,
```

```
        font_weight="bold",
        font_color="#172B4D",
    )
    nx.draw_networkx_edges(
        grafo,
        posiciones,
        edge_color=colores_aristas,
        width=anchos_aristas,
        arrows=True,
        arrowsize=19,
        node_size=2450,
        connectionstyle="arc3,rad=0.04",
        min_source_margin=12,
        min_target_margin=17,
    )
    etiquetas_aristas = {
        (u, v): f"{datos['weight']} {unidad}"
        for u, v, datos in grafo.edges(data=True)
    }
    nx.draw_networkx_edge_labels(
        grafo,
        posiciones,
        edge_labels=etiquetas_aristas,
        font_size=7.5,
        font_color="#253858",
        rotate=False,
        label_pos=0.52,
        bbox={"alpha": 0.88, "color": "white", "pad": 0.9, "linewidth": 0},
    )
    plt.title(titulo, fontsize=13, fontweight="bold", pad=16)
    plt.text(
        0.5,
        0.02,
        "Ruta óptima resaltada en rojo; los números indican el costo de cada tramo.",
        transform=plt.gcf().transFigure,
        ha="center",
        fontsize=8,
    )
    plt.axis("off")
    plt.tight_layout(rect=(0, 0.04, 1, 1))
    plt.savefig(archivo, bbox_inches="tight", facecolor="white")
    plt.close()
def ejecutar_caso(
    nombre: str,
    creador,
    nombre_archivo_grafo: str,
) -> None:
    """Ejecuta el flujo completo: crear, generar estados, buscar y graficar."""
    grafo, etiquetas, posiciones, inicial, meta, unidad = creador()
    ruta, costo, estados = resolver_problema(grafo, inicial, meta)
    print(f"\n=== {nombre} ===")
    print(f"Tipo de grafo: dirigido y ponderado (DiGraph)")
    print(f"Nodos (V): {grafo.number_of_nodes()}")
    print(f"Aristas (E): {grafo.number_of_edges()}")
    print(f"Estados alcanzables desde {inicial}: {', '.join(estados)}")
    print(f"Meta: {meta}")
    print(f"Ruta de menor costo: {' -> '.join(ruta)}")
    print(f"Costo total: {costo} {unidad}")
    archivo = CARPETA_SALIDA / nombre_archivo_grafo
    dibujar_grafo(
        grafo,
        etiquetas,
        posiciones,
        ruta,
        unidad,
        archivo,
        nombre,
    )
    print(f"Diagrama guardado en: {archivo}")
def main() -> None:
```

```
    """Resuelve ambos casos y crea los diagramas para el reporte."""
    CARPETA_SALIDA.mkdir(parents=True, exist_ok=True)
    ejecutar_caso(
        "Caso 1. Ruta de ambulancia (costo: minutos)",
        crear_problema_ambulancia,
        "grafo_ambulancia.png",
    )
    ejecutar_caso(
        "Caso 2. Entrega de artesanías (costo: kilómetros)",
        crear_problema_artesanias,
        "grafo_artesanias.png",
    )
    print("\nEjecución finalizada. Ambos costos se verificaron sumando sus aristas.")
if __name__ == "__main__":
    main()
```

Herramienta utilizada: ChatGPT

Promt:
Resolveré el apartado 2.1 Problemas de Búsqueda y 2.2 Espacios de Búsqueda.
Debo de Modelar 2 problemas reales como acciones, estado, coste, meta.
Debo representar cada espacio como un grafo (dirigido/ponderado). Programar en Python un generador de estados y un grafo con NetworkX.
Debo Entregar el diagrama del grafo, codigo comentado y mostrar un analisis de complejidad.
Segun los dos problemas que investigaste realiza esas acciones por favor.

Uso:
Investigacion, redaccion de texto, creacion de codigo, creacion de esquemas.

Verificacion: El codigo funciona de manera correcta, los diagramas coinciden con el codigo otorgado, fuentes confiables.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271. https://doi.org/10.1007/BF01386390
Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught, & J. Millman (Eds.), Proceedings of the 7th Python in Science Conference (pp. 11–15). https://doi.org/10.25080/tcwv9851

NetworkX Developers. (s. f.-a). DiGraph—Directed graphs with self loops. NetworkX. https://networkx.org/documentation/stable/reference/classes/digraph.html
NetworkX Developers. (s. f.-b). Dijkstra's algorithm. NetworkX. https://networkx.org/documentation/stable/reference/algorithms/shortest_paths/dijkstra.html

Rodrigues, G. de A., Junqueira, L., Vieira, J. G. V., da Silva Júnior, O. S., & da Cunha, C. B. (2025). Rural last-mile distribution using a hybrid heuristic-multi-criteria decision-making framework. Expert Systems with Applications, 289, Article 128331. https://doi.org/10.1016/j.eswa.2025.128331

Russell, S. J., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.
Tereshkov, A., Sapundzhi, F., Ivanova, A., & Georgiev, S. (2026). Robust ambulance dispatch using graph-based optimization under traffic uncertainty: A case study of Prague. Engineering Proceedings, 154(1), Article 91. https://doi.org/10.3390/engproc2026154091